

IIS Deployment: Assigning a Static IP Address

User Question

I want to assign IP address to the existing deployment so tell me where all changes are needed

Do i have add the IP address on both front end and back end project?

Answer

Good question — yes, this touches both sides, but the type of change differs. Let me break down exactly what changes where.

Why you'd do this

Binding to a specific IP (like `192.168.1.10` — your machine's LAN IP) instead of "All Unassigned" means the site will only respond to requests hitting that specific network interface. This is useful if you want other devices on your network to access the app via that IP, or if the server has multiple network cards and you want to control which one serves this site.

1. IIS Binding — needed on BOTH sites

For both your frontend (`Dapper_Frontend`) and backend (`Dapper_CRUD`) sites in IIS:

- Right-click the site → Edit Bindings (or do this now while adding, like your screenshot)
- Change IP address dropdown from All Unassigned to `192.168.1.10`
- Keep the Port as is (8081 for frontend, 8080 for backend) — unless you want to change those too
- Click OK

Do this for both sites separately.

2. Frontend code — update the API URL

Since your backend will now be reachable at `http://192.168.1.10:8080` instead of `http://localhost:8080`, update your `EmployeeService.js`:

```
const API_URL = "http://192.168.1.10:8080/api/Employee";
```

Then rebuild (`npm run build`) and redeploy (copy fresh `dist` contents to `C:\inetpub\wwwroot\dapper_frontend`) — same as before, since this URL is baked in at build time.

3. Backend code — update CORS origin

Your frontend will now be accessed via `http://192.168.1.10:8081` instead of `localhost:8081`. Update `Program.cs`:

```
policy.WithOrigins("http://192.168.1.10:8081")
```

Then republish and recycle the `Dapper_CRUD` app pool — same steps as your earlier CORS fix.

4. One more thing: Windows Firewall

If other devices on your network need to reach this (not just the same machine), make sure Windows Firewall allows inbound traffic on ports 8080 and 8081. Otherwise it'll work fine on localhost from the same machine but fail from other devices.

Quick summary table

Change	Where
IIS binding IP	Both sites (frontend + backend) in IIS Manager
API_URL	Frontend code → rebuild → redeploy
CORS WithOrigins	Backend code → republish → recycle pool
Firewall rule	Server-level, if accessing from other devices

Go ahead and change the bindings first, then update and redeploy both projects, and test by browsing to `http://192.168.1.10:8081` instead of `localhost:8081`.

User Follow-up

[This is working:- dapper_react](#)

[This is not:- 192.168.1.10](#)

In short if i click on front end url it is working and getting me the table but if i onlu click on backend url it is not working

Why is this so?

How to fix it?

Answer

This is actually the same "expected 404 on root" situation from earlier — not a real problem. Notice your frontend tab is showing the employee table with real data, which means the backend IS working and reachable at `192.168.1.10:8080` — the frontend just successfully fetched from it.

The only thing failing is browsing directly to `http://192.168.1.10:8080/` (the bare root), which 404s because your API has no route mapped to `/` — same as when you tested `localhost:8080/` earlier and got the same 404, then it worked once you hit `/api/Employee`.

To confirm this is fine, test the actual endpoint:

```
http://192.168.1.10:8080/api/Employee
```

This should return your JSON employee data directly in the browser — proving the backend is fully working on the new IP.

So to summarize: nothing is broken

- ✓ Backend reachable and serving data (proven by your frontend table populating)
- ✓ Frontend reachable and working
- ✗ Root URL `/` on the backend 404s — this is normal/expected behavior for a Web API with no root route defined, not a bug

If you still want / to show something (optional, not required)

You could add a simple root endpoint just so hitting the bare URL doesn't look like an error, e.g. in `Program.cs`:

```
app.MapGet("/", () => "Dapper CRUD API is running");
```

This is purely cosmetic though — not needed for your app to function. Your actual deployment is working correctly as-is.

User Follow-up

Link that we deployed on IIS

Answer

That's this one (frontend, from earlier):

```
http://192.168.1.10:8081
```

Note: this only works from devices on the same local network as the server — since it's bound to the machine's LAN IP, not a public domain. On a different network (e.g. mobile data, another WiFi), it won't be reachable.

User Follow-up

It's not working on my desktop as it uses Ethernet but it's either not working on my mobile also as it is connected on the same wifi

Why?

Answer

Most likely cause: Windows Firewall on the server is blocking inbound traffic on ports 8080/8081, since by default Windows only allows local traffic and blocks unsolicited inbound connections from the network.

Fix 1: Add firewall rules for the ports

On the server machine:

- Open Windows Defender Firewall with Advanced Security (search in Start menu)
- Click Inbound Rules (left panel) → New Rule (right panel)
- Select Port → Next
- TCP, Specific local ports: 8080,8081 → Next
- Allow the connection → Next
- Check all profiles (Domain, Private, Public) → Next
- Name it something like "IIS Dapper App" → Finish

Fix 2: Check the network profile

If your server's network connection (Ethernet/WiFi) is set to Public instead of Private, Windows applies stricter firewall rules by default, sometimes blocking things even with a rule in place.

- Go to Settings → Network & Internet
- Click your active connection
- Set Network profile to Private (if it's currently Public)

Fix 3: Confirm the IP is actually correct and current

Since your desktop is on Ethernet, not WiFi, double-check `192.168.1.10` is still the server's correct current IP — it may have changed if it's DHCP-assigned rather than static. On the server, run:

```
ipconfig
```

and confirm the IPv4 address under the active adapter still matches 192.168.1.10.

After adding the firewall rule

Test again from another device on the same WiFi:

```
http://192.168.1.10:8081
```

Let me know what happens — if it still fails, we'll check whether it's a firewall issue or something else (like router-level client isolation on the WiFi network, which some routers enable by default and blocks devices from seeing each other even on the same network).